

Guía de Trabajo para Proyectos de Análisis Numérico

(Con Git local, entornos virtuales, testing y CI/CD)

1. Introducción

El propósito de este documento es establecer una metodología de trabajo robusta, reproducible y profesional para desarrollar proyectos de análisis numérico. Aunque el contexto inicial es el estudio autodidacta, las prácticas aquí descritas son las mismas que se utilizan en equipos de investigación, desarrollo de software científico e ingeniería de alto nivel.

La metodología se apoya en tres pilares:

1. **Control de versiones con Git** – para registrar el proceso, experimentar sin miedo y mantener un historial comprensible.
2. **Entornos virtuales aislados** – para garantizar reproducibilidad y evitar conflictos entre dependencias.
3. **Pruebas automatizadas e integración continua** – para validar que los algoritmos se comportan como se espera y detectar regresiones temprano.

Todo esto se aplicará **sin necesidad de un repositorio remoto inicial**, aunque se explicará cómo añadirlo después si se desea.

2. Filosofía de trabajo

La filosofía que guía esta metodología se resume en:

“Cada cambio debe ser pequeño, comprensible y reversible.”

- **Commits atómicos y significativos:** cada commit representa un paso lógico completo (por ejemplo, “añade función de Newton”, “corrige condición de parada”, “documenta ejemplo”). No mezcles cambios no relacionados.
- **Ramas para experimentación:** nunca trabajes directamente sobre la rama principal (`main`) cuando estés desarrollando una nueva característica o explorando una idea. Usa ramas de características (`feature/`) que luego fusionarás.
- **Reproducibilidad:** el entorno de desarrollo debe poder recrearse en cualquier máquina con un solo comando. Esto implica definir explícitamente todas las dependencias y su versión.
- **Pruebas como documentación ejecutable:** las pruebas unitarias no solo verifican que el código funciona; también documentan el comportamiento esperado de las funciones y sirven como ejemplo de uso.

- **Código como narrativa:** organiza el código en módulos reutilizables, separa la lógica de la exploración (notebooks vs. scripts) y documenta con comentarios y docstrings.

3. Control de versiones con Git: conceptos fundamentales y flujo de trabajo

Git gestiona el historial de cambios mediante un modelo basado en **tres árboles**: el directorio de trabajo (working directory), el área de staging (index) y el repositorio local (commit history). Entender esta distinción es clave para usar Git correctamente.

3.1 Ciclo de vida de un archivo en Git

1. **Untracked:** archivo nuevo que Git aún no controla.
2. **Staged (indexado):** archivo marcado para ser incluido en el próximo commit.
3. **Committed:** archivo cuyo estado ha sido registrado en la historia del repositorio.

El flujo básico es:

Working Directory → git add → Staging Area → git commit → Repository (local)

3.2 El área de staging (index)

El área de staging es una zona intermedia donde preparas los cambios que formarán parte del próximo commit. **No es obligatorio agregar todos los archivos modificados:** puedes seleccionar exactamente qué cambios quieres incluir, incluso partes de un mismo archivo (con `git add -p`). Esto permite construir commits atómicos y coherentes.

Comandos básicos para mover archivos entre áreas:

Acción	Comando
Ver estado actual	<code>git status</code>
Añadir archivo al staging	<code>git add archivo</code>
Añadir todos los archivos modificados/creados	<code>git add .</code>
Añadir solo archivos modificados (no nuevos)	<code>git add -u</code>
Sacar archivo del staging (pero mantener cambios en working directory)	<code>git reset archivo</code>
Sacar archivo del staging y descartar cambios (volver al último commit)	<code>git checkout -- archivo</code>
Ver diferencias entre working directory y staging	<code>git diff</code>
Ver diferencias entre staging y último commit	<code>git diff --staged</code>

3.3 El commit: fijando un estado

Un commit captura el estado actual del staging area junto con un mensaje descriptivo. Una vez hecho, ese estado queda registrado permanentemente en la historia local.

Bash

```
git commit -m "tipo: descripción clara"
```

Si olvidaste incluir algún archivo, puedes agregarlo y luego hacer `git commit --amend` para modificar el último commit (útil solo si aún no has compartido ese commit).

3.4 Ramas (branches)

Las ramas permiten desarrollar características de forma aislada. La rama principal por defecto suele llamarse `main` (antes `master`).

bash

```
git branch                # lista ramas locales
git checkout -b feature/nueva # crea y cambia a nueva rama
git checkout main          # vuelve a la rama principal
git merge feature/nueva    # integra los cambios de la rama en la actual
```

3.5 Flujo de trabajo recomendado para desarrollo local

1. Asegúrate de estar en `main` y con el repositorio limpio (`git status`).
2. Crea una rama para la nueva funcionalidad: `git checkout -b feature/metodo-newton`.
3. Trabaja en los cambios (edita archivos, crea pruebas, etc.).
4. Revisa qué has cambiado: `git status` y `git diff`.
5. Agrega al staging los archivos que correspondan (puedes usar `git add -p` para seleccionar fragmentos).
6. Haz commit: `git commit -m "feat: implementa método de Newton con derivada"`.
7. Repite los pasos 3-6 cuantas veces sea necesario (cada commit debe ser un paso lógico).
8. Cuando la funcionalidad esté completa, vuelve a `main`: `git checkout main`.
9. Fusiona la rama: `git merge feature/metodo-newton`.
10. Elimina la rama si ya no la necesitas: `git branch -d feature/metodo-newton`.

Este flujo mantiene `main` siempre estable y te permite experimentar sin riesgos.

4. Uso de Git sin GitHub (control de versiones local)

Git funciona perfectamente sin necesidad de un servidor remoto. Toda la historia se almacena en la carpeta `.git` de tu proyecto. Esta es la forma más simple de empezar.

4.1 Inicializar un repositorio local

```
bash
```

```
cd mi_proyecto_analisis_numerico
git init
```

Esto crea un repositorio vacío en el directorio actual. A partir de ahora, todos los cambios que registres quedarán guardados localmente.

4.2 Recomendaciones para commits locales

- No esperes a tener todo el proyecto listo para hacer commit. Haz commits frecuentes, incluso varias por hora.
- Cada commit debe dejar el proyecto en un estado “funcional” (que al menos los tests pasen, si los hay). No hagas commit con código roto.
- Usa mensajes en presente imperativo (“agrega”, “corrige”, “refactoriza”, no “agregado” ni “corregí”).

Ejemplo de historial local limpio:

```
text
```

```
* abc1234 (HEAD -> main) docs: actualiza README con instrucciones de entorno
* def5678 test: añade pruebas de convergencia para método de Newton
* ghi9012 feat: implementa método de Newton con análisis de error
* jkl3456 chore: inicializa estructura de proyecto y entorno conda
```

Si más adelante decides compartir tu trabajo, puedes conectar este repositorio local con un remoto (GitHub, GitLab) usando:

```
bash
```

```
git remote add origin https://github.com/tuusuario/mi_proyecto.git
git push -u origin main
```

Pero mientras tanto, trabajar localmente es perfectamente válido.

5. Metodología de trabajo: desde la creación del entorno hasta la estructura

5.1 Estructura de directorios recomendada

text

```
mi_proyecto_analisis_numerico/
├── .git/
├── .gitignore
├── README.md
├── environment.yml          # Definición del entorno Conda (para Python)
├── src/                    # Código fuente reutilizable
│   ├── __init__.py
│   ├── root_finding.py
│   ├── linear_algebra.py
│   └── utils.py
├── tests/                  # Pruebas unitarias
│   ├── test_root_finding.py
│   ├── test_linear_algebra.py
│   └── confest.py          # Configuración de pytest (fixtures, etc.)
├── notebooks/              # Jupyter notebooks para exploración
│   ├── 01_intro.ipynb
│   └── ...
├── data/                   # Datos (opcional, puede estar en .gitignore)
│   └── .gitkeep
├── scripts/                # Scripts ejecutables (por ejemplo, para generar
│   └── plot_convergence.py  gráficos)
└── docs/                   # Documentación adicional (opcional)
```

5.2 Entornos mixtos: Python y Julia trabajando juntos

Cuando un proyecto requiere tanto Python como Julia, es importante mantener la reproducibilidad y evitar conflictos. La mejor práctica es:

- **Usar Conda para Python** (y opcionalmente como gestor de instalación de Julia), pero **usar el gestor nativo de Julia (Pkg) para los paquetes de Julia**.
- **No mezclar entornos:** activa el entorno Conda para Python y, dentro de él, utiliza Julia con su propio entorno de proyecto.

Opción A: Julia instalado por separado (recomendada)

1. Instala Julia con `juliaup` (o desde la página oficial).
2. Crea un entorno Conda solo para Python con tus dependencias.
3. En tu proyecto, además del `environment.yml`, crea un `Project.toml` y `Manifest.toml` para Julia.
4. Documenta en el README cómo instalar Julia y activar el entorno de Julia (`julia --project=.`).

Opción B: Instalar Julia a través de Conda

Aunque es posible (`conda install -c conda-forge julia`), puede generar versiones no actualizadas y no integra bien el sistema de paquetes de Julia. Si decides hacerlo, recuerda que

luego deberás gestionar los paquetes de Julia con `Pkg` dentro del entorno Conda, y puede haber conflictos de rutas.

Ejemplo de estructura para proyecto mixto:

text

```
mi_proyecto_mixto/
├── environment.yml          # Entorno Conda para Python
├── Project.toml             # Entorno de Julia
├── Manifest.toml           # (opcional, se genera automáticamente)
├── src/
│   ├── python/             # Módulos Python
│   │   └── ...
│   ├── julia/              # Módulos Julia
│   │   └── ...
├── notebooks/              # Notebooks (pueden usar PyCall o Julia)
└── tests/                  # Pruebas separadas para cada lenguaje
```

Comunicación entre Python y Julia:

- Desde Julia, usar `PyCall` para llamar a código Python.
- Desde Python, usar `juliacall` (de la librería `julia`) para invocar funciones Julia.

Para que `PyCall` use el entorno Conda activo, debes configurarlo:

julia

```
ENV["PYTHON"] = "/ruta/al/ejecutable/python/del/entorno/conda"
using Pkg
Pkg.build("PyCall")
```

Recomendación: si tu proyecto es mayoritariamente en Python y solo usas Julia para algunas tareas específicas, gestiona Julia por separado y documenta los pasos de instalación. Si es un proyecto puramente en Julia, no uses Conda para Julia; utiliza el propio gestor de Julia.

5.3 Configuración de Git y `.gitignore`

Crea un archivo `.gitignore` adecuado para tu proyecto. Para Python + Jupyter + Julia, un buen punto de partida es:

text

```
# Python
__pycache__/
*.py[cod]
*.so
*.egg-info/
dist/
build/
.env/
.venv/

# Julia
*.jl.cov
*.jl*.cov
*.jl.mem
Manifest.toml # Opcional: se puede versionar para fijar dependencias exactas
/Julia/
```

```

LocalPreferences.toml

# Notebooks (limpiar salidas)
.ipynb_checkpoints/
*.ipynb
!notebooks/*.ipynb # Excepto los que queremos versionar
# Si usas nbstripout, los notebooks pueden versionarse sin salidas

# Datos grandes
data/
*.h5
*.npz

# Archivos de sistema
.DS_Store
Thumbs.db

```

Recomendación: para versionar notebooks sin salidas, usa nbstripout:

```

bash

pip install nbstripout
nbstripout --install # Instala el hook para limpiar notebooks automáticamente
al commit

```

5.4 Creación del entorno Conda (para Python)

Ejemplo de environment.yml:

```

yaml

name: analisis_numerico
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.10
  - numpy=1.24
  - scipy=1.10
  - matplotlib=3.7
  - jupyterlab=4.0
  - pytest=7.4
  - pip
  - pip:
    - -e . # Instala el paquete local (src/) en modo desarrollo

```

Para crear el entorno:

```

bash

conda env create -f environment.yml
conda activate analisis_numerico

```

5.5 README.md esencial

El README.md debe contener al menos:

- Título y descripción breve del proyecto.
- Instrucciones para crear el entorno (conda env create -f environment.yml).

- Si hay código Julia, instrucciones para instalar Julia y activar el entorno (`julia --project=.`).
 - Cómo ejecutar las pruebas (pytest, o `julia --project=. -e 'using Pkg; Pkg.test()'.`
 - Un ejemplo mínimo de uso.
-

6. Pruebas (testing) y CI/CD

6.1 Filosofía de pruebas para análisis numérico

En análisis numérico, las pruebas deben verificar no solo que el código no lanza errores, sino que los resultados son correctos dentro de tolerancias razonables. Por tanto:

- **Pruebas de convergencia:** comprobar que un método iterativo alcanza la solución esperada después de un número conocido de iteraciones, o que el error disminuye según el orden teórico.
- **Pruebas de estabilidad:** usar matrices mal condicionadas (por ejemplo, la matriz de Hilbert) y verificar que el algoritmo se comporta de forma esperada (quizás fallando de manera controlada).
- **Pruebas de consistencia:** comparar con resultados de librerías consolidadas (scipy, numpy.linalg) para validar implementaciones propias.

6.2 Herramientas y estructura de pruebas

Usaremos **pytest** para Python. Para Julia, el propio **Test** estándar.

Ejemplo de archivo `tests/test_root_finding.py`:

python

```
import numpy as np
import pytest
from src.root_finding import newton_raphson

def test_newton_raphson_converges():
    f = lambda x: x**2 - 4
    df = lambda x: 2*x
    root, iterations = newton_raphson(f, df, x0=1.0, tol=1e-10)
    assert abs(root - 2.0) < 1e-10
    assert iterations > 0

def test_newton_raphson_derivative_zero():
    f = lambda x: (x-1)**3
    df = lambda x: 3*(x-1)**2
    with pytest.raises(ValueError, match="Derivada cercana a cero"):
        newton_raphson(f, df, x0=1.0)
```

Organización:

- Pon todas las pruebas dentro de la carpeta `tests/`.

- Usa `conftest.py` para fixtures comunes (ej. matrices de prueba).
- Nombra los archivos de prueba con prefijo `test_`.

Ejecutar pruebas localmente:

```
bash
pytest tests/
```

6.3 Integración continua (CI) / Entrega continua (CD)

Aunque trabajes localmente, es recomendable configurar un servicio de CI para que cada vez que hagas commit (o push) se ejecuten automáticamente las pruebas en un entorno limpio. Esto te asegura que no introduces errores inadvertidos.

¿Cómo hacerlo sin GitHub?

Puedes usar sistemas de CI locales (como `act` para simular GitHub Actions) o simplemente adoptar la disciplina de ejecutar `pytest` antes de cada commit. Pero lo más profesional es usar un repositorio remoto (GitHub/GitLab) y aprovechar su CI gratuito. Dado que en esta guía priorizamos el trabajo local, al menos te recomiendo:

1. Configurar un **hook de pre-commit** que ejecute `pytest` y cancele el commit si falla.
2. Si en el futuro subes el repositorio a GitHub, añadir un archivo `.github/workflows/ci.yml` que ejecute las mismas pruebas en la nube.

Ejemplo de pre-commit hook simple (archivo `.git/hooks/pre-commit`):

```
bash
#!/bin/sh
pytest
if [ $? -ne 0 ]; then
    echo "Pruebas fallidas. No se realiza el commit."
    exit 1
fi
```

Recuerda darle permisos de ejecución: `chmod +x .git/hooks/pre-commit`.

Para proyectos mixtos (Python + Julia), podrías tener un script que ejecute las pruebas de ambos lenguajes.

6.4 Documentación de pruebas

Incluye en el README cómo ejecutar las pruebas y qué cobertura esperas. Si usas `pytest-cov` puedes generar reportes de cobertura.

7. Conclusión

La metodología descrita transforma el estudio del análisis numérico en una práctica profesional. Al aplicar Git localmente con ramas, commits semánticos y una estructura ordenada, tendrás un control total sobre tu progreso. El uso de Conda (y eventualmente Julia con su propio gestor) garantiza la reproducibilidad. Las pruebas automatizadas te darán confianza en que tus algoritmos funcionan correctamente, y los hooks de CI locales te mantendrán disciplinado.

Si en algún momento decides compartir tu trabajo, todo estará listo para publicarlo en un repositorio remoto con un historial limpio y una documentación sólida.

Nota final: Este documento es un punto de partida. Adapta la estructura y herramientas a tus necesidades específicas, pero mantén la filosofía de **control, reproducibilidad y validación continua**.

Si necesitas ampliar algún punto (por ejemplo, cómo configurar un pre-commit hook con herramientas como `black`, `flake8`, o cómo integrar pruebas de Julia en el mismo flujo), podemos seguir refinando.